

# СЪДЪРЖАНИЕ

---

|     |                                                                    |    |
|-----|--------------------------------------------------------------------|----|
| 1.  | Увод .....                                                         | 3  |
| 2.  | Теоретични основи и анализ на технологиите .....                   | 4  |
| 2.1 | Развитие на системите за дистанционно управление в роботиката..... | 4  |
| 2.2 | Безжични комуникационни стандарти и Bluetooth технология .....     | 5  |
| 2.3 | Микроконтролерни платформи за вградени системи.....                | 6  |
| 2.4 | Уеб технологии и клиент-сървърна архитектура в роботиката .....    | 7  |
| 2.5 | Кинематика на мобилните роботи и управление с приплъзване .....    | 8  |
| 2.6 | Теоретични аспекти на широчинно-импулсната модулация.....          | 9  |
| 2.7 | Принципи на асинхронната серийна комуникация (UART).....           | 10 |
| 3.  | Апаратно осигуряване (хардуерна реализация) .....                  | 12 |
| 3.1 | Изчислителен и управляващ блок .....                               | 12 |
| 3.2 | Задвижваща система и управление на двигатели .....                 | 12 |
| 3.3 | Енергийна система и захранване .....                               | 14 |
| 3.4 | Комуникационен интерфейс и обща електрическа схема .....           | 14 |
| 4.  | Програмно осигуряване и реализация на системата.....               | 16 |
| 4.1 | Архитектура на софтуерната система.....                            | 16 |
| 4.2 | Програмиране на микроконтролера (Arduino).....                     | 17 |
| 4.3 | Разработка на бекенд системата с Django.....                       | 20 |
| 4.4 | Проектиране на потребителския уеб интерфейс (Frontend) .....       | 22 |

|     |                                                       |    |
|-----|-------------------------------------------------------|----|
| 5.  | Тестване, анализ и резултати .....                    | 25 |
| 5.1 | Методика на тестване .....                            | 25 |
| 5.2 | Анализ на надеждността и закъсненията (Latency) ..... | 26 |
| 5.3 | Оценка на цялостното представяне.....                 | 27 |
| 6.  | Заключение .....                                      | 29 |
| 7.  | Използвана литература и източници .....               | 31 |

# 1. УВОД

---

В съвременния технологичен свят границите между дигиталното пространство и физическата реалност стават все по-незабележими. Развитието на концепцията за Интернет на нещата (Internet of Things – IoT) и напредъкът в областта на вградените системи превърнаха дистанционното управление на физически обекти чрез софтуерни интерфейси от специализирана индустриална практика в достъпна и широко приложима технология. Мобилната роботика, като неизменна част от този процес, намира приложение в редица сфери – от автоматизирани складови системи и изследователски мисии в опасни среди, до образователни платформи и интелигентни домашни асистенти.

Традиционните методи за управление на роботизирани платформи дълго време разчитаха на специализирани радиочестотни контролери (RC), които, макар и надеждни, ограничават гъвкавостта на системата и изискват наличието на специфичен хардуер у потребителя. Днес тенденцията е към унифициране на контролните интерфейси чрез използване на вече съществуващи устройства, като смартфони, таблети и персонални компютри, посредством стандартни уеб браузъри. Този подход не само демократизира достъпа до управление на роботи, но и позволява интеграцията им в по-големи, комплексни информационни системи.

В този контекст се оформя и основната цел на настоящия дипломен проект: проектиране, сглобяване и софтуерно осигуряване на дистанционно управлявана мобилна роботизирана платформа, която приема команди в реално време чрез интуитивен уеб интерфейс. Системата е замислена като хибридно решение, което съчетава надеждността на микроконтролерното управление на хардуерно ниво с гъвкавостта на съвременните уеб технологии на софтуерно ниво.

За успешното реализиране на тази основна цел, в рамките на проекта са дефинирани и изпълнени следните конкретни задачи:

Първо, да се изгради стабилно и симетрично хардуерно шаси, задвижвано от четири постояннотокови (DC) двигателя, което да гарантира висока проходимост и маневреност. Второ, да се реализира надеждна енергийна система, базирана на 18650

литиево-йонни батерии и подходящо преобразуване на напрежението, за да се защити управляващата електроника от смущения. Трето, да се разработи програмен код за микроконтролерна платка (Arduino), който да обработва входящите сигнали и прецизно да управлява моторния драйвер. Четвърто, да се изгради комуникационен мост между физическия робот и управляващия компютър чрез използване на Bluetooth технология. И накрая, но не на последно място по важност, да се програмира пълноценно сървърно приложение (базирано на платформата Django), което да предоставя графичен потребителски интерфейс в уеб браузъра и да транслира потребителските команди (напред, назад, наляво, надясно, стоп) в хардуерни инструкции.

## **2. ТЕОРЕТИЧНИ ОСНОВИ И АНАЛИЗ НА ТЕХНОЛОГИИТЕ**

---

### **2.1 РАЗВИТИЕ НА СИСТЕМИТЕ ЗА ДИСТАНЦИОННО УПРАВЛЕНИЕ В РОБОТИКАТА**

Еволюцията на дистанционното управление при мобилните роботи преминава през няколко ключови етапа, всеки от които е продиктуван от нуждите за по-голям обхват, по-висока надеждност и по-богата функционалност. В началните етапи на роботиката управлението се е осъществявало предимно чрез кабелна връзка (tethered control). Този метод осигурява нулева латентност и непрекъснато захранване, но физическият кабел драстично ограничава обхвата на движение и прави системата неприложима за автономни изследователски мисии или работа в сложна и пресечена среда.

Следващата голяма стъпка е навлизането на аналоговите и впоследствие цифровите радиочестотни (RC) системи. Те премахват физическото ограничение на кабела и позволяват управление на значителни разстояния. Въпреки това, RC системите изискват специализиран предавател и приемник, често обвързани с конкретен производител или протокол. Потребителят е длъжен да носи допълнителен, специализиран хардуер – дистанционно управление, което не предлага възможности за лесно надграждане или интеграция с други софтуерни системи.

Съвременният етап от развитието на тези системи се характеризира с преминаване към стандартизирани безжични мрежи с общо предназначение – като Wi-Fi и Bluetooth. Тази парадигма променя изцяло архитектурата на управлението. Вместо роботът да комуникира със специализиран пулт, той се превръща във възел от локална или глобална мрежа. Това позволява контролът да се абстрахира и да се изнесе на софтуерно ниво. Управлението вече не изисква специфични джойстици; то може да се осъществи от всяко устройство, разполагащо с екран и интернет браузър. Именно тази концепция стои в основата на настоящия проект – създаване на система, при която физическото движение в реалния свят е пряк резултат от взаимодействие с виртуален интерфейс.

## **2.2 Безжични комуникационни стандарти и Bluetooth технология**

За осъществяване на връзката между управляващия сървър и роботизираната платформа е необходимо избирането на подходящ безжичен стандарт. В света на вградените системи съществуват различни алтернативи, всяка със своите предимства и недостатъци. За целите на този проект, изискващ надеждна комуникация на кратки до средни разстояния с ниска консумация на енергия, технологията Bluetooth се явява оптималното решение.

Bluetooth представлява глобален стандарт за безжична комуникация, първоначално проектиран да замени кабелните връзки при обмяна на данни между портативни устройства. Технологията оперира в нелицензираната ISM (Industrial, Scientific, and Medical) честотна лента, позиционирана около 2.4 GHz. Този честотен спектър е глобално достъпен, което прави стандарта универсален. Комуникацията се осъществява чрез техника на скокообразно изменение на честотата (Frequency-Hopping Spread Spectrum – FHSS), при която носещата честота се променя стотици пъти в секунда. Този подход драстично намалява влиянието на външни интерференции от други устройства, работещи в същия обхват (като например Wi-Fi рутери), и повишава общата сигурност на предаването на данни.

Стандартната спецификация на Bluetooth включва различни класове на мощност, които определят ефективния обхват. В типичните приложения за мобилна роботика на закрито се разчита на обхват до около 10 метра, което е напълно

достатъчно за осигуряване на непрекъснат визуален контакт и контрол над машината. Освен това стандартът въвежда протоколи за криптиране и автентикация, защитавайки системата от неоторизиран достъп.

В рамките на хардуерната реализация на този проект, ключова роля играе модулът HC-05. Този компонент е изключително популярен в инженерните и хоби средите поради своята достъпност и изключително лесна интеграция. За разлика от някои по-ограничени модули, HC-05 има способността да работи както в режим на подчинено устройство (slave), така и като главно устройство (master). В контекста на нашия проект, той е конфигуриран да приема връзки от главния управляващ компютър.

Модулът действа като прозрачен мост – той приема безжичните Bluetooth пакети от сървъра и ги преобразува в стандартни серийни (UART) сигнали, които микроконтролерът може лесно да разчете. Въпреки неоспоримите си предимства като отлична документация и лесна конфигурация чрез AT команди, използването на HC-05 крие и някои предизвикателства. Едно от тях е електрическата съвместимост – логическите нива на модула често изискват внимателно съгласуване на напреженията (например 3.3V спрямо 5V на микроконтролера), за да се избегне повреда на електрониката. Друг фактор, който трябва да се вземе предвид при проектирането на системата за управление, е латентността. Макар и минимални, съществуват времеви закъснения при преноса и декодирането на сигнала. В архитектури, изискващи абсолютно реално време (милисекунди реакция), тази латентност трябва да се компенсира софтуерно, макар за нуждите на настоящата платформа скоростите на предаване да са напълно задоволителни.

## **2.3 МИКРОКОНТРОЛЕРНИ ПЛАТФОРМИ ЗА ВГРАДЕНИ СИСТЕМИ**

Сърцето на всяка съвременна роботизирана платформа е нейният изчислителен блок. Докато персоналните компютри и сървърите са проектирани да изпълняват множество сложни задачи едновременно под управлението на тежки операционни системи, вградените системи изискват различен подход. Те се нуждаят от хардуер, който е оптимизиран за изпълнение на конкретна задача с висока предвидимост, ниска консумация на енергия и най-вече – способност за директно взаимодействие с физически електрически компоненти като сензори и двигатели.

За реализацията на управлението на ниско ниво в проекта е избран микроконтролер от екосистемата на Arduino. Arduino не е просто отделен чип, а цялостна платформа с отворен код, съчетаваща хардуерни платки и удобна софтуерна среда за разработка (IDE). Основното предимство на тази архитектура за целите на мобилната роботика е наличието на богат набор от входно-изходни пинове (GPIO - General Purpose Input/Output). Чрез тях микроконтролерът може да генерира ШИМ (Широчинно-импулсна модулация / PWM) сигнали, които са критично важни за плавното регулиране на оборотите на постояннотоковите двигатели.

Освен това, микроконтролерът е натоварен със задачата да следи непрекъснато серийния порт за входящи данни от Bluetooth модула. При получаване на конкретен символ или низ (например команда за движение напред), софтуерът в микроконтролера мигновено интерпретира тази команда и променя логическите нива на съответните изходи, свързани към драйвера на моторите. Липсата на сложна операционна система при Arduino гарантира, че този цикъл на четене и реакция се случва с микросекундна точност, без прекъсвания от фонові процеси, което е жизненоважно за безопасното и прецизно управление на робота.

## **2.4 УЕБ ТЕХНОЛОГИИ И КЛИЕНТ-СЪРВЪРНА АРХИТЕКТУРА В РОБОТИКАТА**

Управлението на физически обект чрез уеб браузър изисква изграждането на многослойна софтуерна архитектура, която да преведе намеренията на потребителя (кликване на бутон на екрана) в електрически импулс към моторите. Тази парадигма се опира на класическия модел "клиент-сървър".

В тази екосистема, "клиентът" е уеб браузърът на потребителя. В него се зарежда визуалният интерфейс (Frontend), изграден чрез маркиращи езици и скриптове (HTML, CSS и JavaScript). Когато потребителят взаимодейства с интерфейса – например задържи виртуалния бутон за движение напред – JavaScript логиката регистрира това събитие и генерира мрежова заявка (HTTP request). Важна особеност тук е използването на асинхронни технологии, които позволяват изпращането на команди без да се налага презареждане на цялата уеб страница, което би направило управлението накъсано и неефективно.

Тази мрежова заявка пътува до "сървър" (Backend). В настоящия проект сървърната част е реализирана с помощта на Django – мощна уеб рамка (framework), написана на програмния език Python. Django сървърът не е инсталиран на самия робот, а работи на компютър, който действа като централен диспечер. Ролята на сървъра е да приеме уеб заявката, да я валидира и да вземе решение какво действие трябва да се предприеме.

Тук се случва магията на интеграцията: след като Django разпознае командата за движение, сървърното приложение използва системните ресурси на компютъра (чрез Python библиотеки), за да отвори виртуалния сериен порт, свързан с Bluetooth антената на компютъра. Сървърът изпраща съответния кратък текстови код (например буквата 'F' за Forward) през Bluetooth ефира. Оттам сигналят се улавя от HC-05 модула на робота, предава се на микроконтролера, и цикълът завършва с физическото завъртане на колелата.

Този архитектурен подход осигурява огромна гъвкавост. Потребителският интерфейс може лесно да бъде променян, визуално обогатяван или превеждан на различни езици, без да се налага препрограмиране на самия робот. Цялата тежка изчислителна логика остава на сървъра, докато роботът действа като бърз и ефективен изпълнител на кратки команди.

## **2.5 КИНЕМАТИКА НА МОБИЛНИТЕ РОБОТИ И УПРАВЛЕНИЕ С ПРИПЛЪЗВАНЕ**

При проектирането на мобилни роботизирани платформи, изборът на кинематичен модел определя изцяло маневреността и сложността на управлението на машината. В класическата автомобилна индустрия се използва кинематиката на Акерман (Ackermann steering), при която предните колела се завъртат на определен ъгъл чрез механична рейка. В мобилната роботика обаче, стремежът към механична простота и способност за завиване в тесни пространства налага използването на нехолономни системи (системи с ограничения в посоката на движение), най-популярните от които са диференциалното управление (Differential Drive) и управлението с приплъзване (Skid-Steering).

Настоящият проект използва архитектура с четири фиксирани задвижващи колела, което класифицира системата като Skid-Steering платформа (подобно на



верижните машини или мини-товарачите тип "Bobcat"). При тази кинематика колелата от лявата и дясната страна на робота са механично независими едни от други, но колелата от една и съща страна обикновено се въртят с еднаква скорост и в еднаква посока.

Завиването при този модел не се осъществява чрез промяна на ъгъла на осите, а чрез създаване на разлика в ъгловите скорости на лявата и дясната двойка колела. Когато левите двигатели се въртят напред по-бързо от десните, платформата се завърта надясно. Най-голямото предимство на Skid-Steering кинематиката е възможността за завъртане на място (point turn или zero-radius turn). За да се постигне това, левите колела се завъртат в посока напред, а десните – в посока назад (или обратно). В този момент възниква двоица сили, която създава въртящ момент около геометричния център на тежестта на робота.

Теоретичното предизвикателство при Skid-Steering управлението е триенето. Тъй като колелата са фиксирани и не могат да променят ъгъла си спрямо настилка, по време на завой те неизбежно трябва да се приплъзват странично (skid) върху повърхността. Това странично приплъзване генерира значително съпротивление и изисква голям въртящ момент от двигателите. Именно това теоретично съображение оправдава използването на четири отделни постояннотокови (DC) двигателя с редуктори в нашата хардуерна реализация, тъй като те могат да осигурят необходимата тяга за преодоляване на силите на триене при завиване на място.

## **2.6 ТЕОРЕТИЧНИ АСПЕКТИ НА ШИРОЧИННО-ИМПУЛСНАТА МОДУЛАЦИЯ**

Управлението на скоростта и посоката на постояннотоковите (DC) двигатели е фундаментален проблем в роботиката. Тъй като микроконтролерите (като Arduino) са цифрови устройства, те могат да подават на изходите си само две логически състояния – логическа нула (0V) или логическа единица (например 5V). Те не могат да генерират аналогово напрежение (например 2.5V), за да накарат един мотор да се върти наполовина от максималната си скорост. Решението на този проблем е технологията, наречена Широчинно-импулсна модулация (ШИМ или PWM – Pulse Width Modulation).

ШИМ представлява метод за симулиране на аналогово напрежение чрез бързо превключване на цифровия сигнал между състояния "включено" и "изключено".

Основният параметър на ШИМ сигнала е неговият коефициент на запълване (Duty Cycle), който се изразява в проценти. Той показва каква част от времето на един период сигналът е във високо състояние. Например, ако сигналът е включен през 50% от времето и изключен през останалите 50%, средното ефективно напрежение, което моторът "усеща", ще бъде половината от максималното. Тъй като двигателите имат голяма механична инерция и индуктивност, те не реагират на отделните импулси с накъсано движение, а интегрират енергията и се въртят плавно със скорост, пропорционална на коефициента на запълване.

Докато ШИМ решава проблема със скоростта, контролът на посоката се осъществява чрез електронна топология, известна като Н-мост (H-Bridge). Теоретичният модел на Н-моста се състои от четири електронни ключа (обикновено биполарни или полеви транзистори), подредени схематично под формата на буквата "Н", с постояннотоковия мотор в средата на хоризонталната връзка.

Логиката на работа е следната: когато се затворят два диагонално разположени ключа (например горе вляво и долу вдясно), токът протича през мотора от ляво надясно и той се върти напред. Ако тези ключове се отворят и се затвори другата диагонална двойка (горе вдясно и долу вляво), посоката на тока през мотора се обръща, което води до завъртане на ротора в обратна посока.

Критичен момент в теорията на Н-мостовете е предотвратяването на т.нар. "прострелващ ток" (shoot-through). Ако двата ключа от една и съща страна (горен и долен) се затворят едновременно, се получава директно късо съединение между захранването и земята, което води до мигновено разрушаване на компонентите. Интегрираните драйвери, като използвания в проекта L293D, съдържат вътрешна хардуерна логика, която прави такова състояние невъзможно, осигурявайки безопасна и надеждна работа на системата.

## **2.7 ПРИНЦИПИ НА АСИНХРОННАТА СЕРИЙНА КОМУНИКАЦИЯ (UART)**

Преносът на команди от Bluetooth модула към изчислителния блок на работа се реализира посредством асинхронна серийна комуникация, управлявана от хардуерен модул, наречен UART (Universal Asynchronous Receiver-Transmitter). Разбирането на

теоретичните принципи на този протокол е ключово за анализа на комуникационната надеждност на системата.

За разлика от паралелната комуникация, при която множество битове се предават едновременно по отделни проводници, серийната комуникация предава данните бит по бит последователно по една единствена линия (TX за предаване и RX за приемане). Терминът "асинхронна" означава, че между предавателя и приемника не съществува обща тактова линия (clock signal), която да синхронизира четенето на данните. Вместо това, двете устройства трябва да бъдат предварително настроени да работят с еднаква скорост на предаване (Baud Rate), измервана в битове в секунда (bps). В нашия случай стандартната скорост е 9600 bps.

Тъй като липсва тактов сигнал, протоколът използва специални времеви маркери, за да укаже началото и края на всеки пакет от данни (обикновено един байт или 8 бита). Когато комуникационната линия е в покой, тя се поддържа във високо логическо състояние (логическа 1). Когато предавателят (в случая HC-05) иска да изпрати команда, той първо сваля линията в ниско състояние за продължителността на един бит. Това е т.нар. "Старт бит" (Start bit). Той "събужда" приемника (Arduino) и му сигнализира, че следващите импулси ще съдържат полезни данни.

След старт бита следват самите данни (обикновено 8 бита, представляващи ASCII кода на командата, например 'F' за напред). Те се предават от най-младшия към най-старшия бит. След приключване на данните, линията се връща във високо състояние за минимум един времеви интервал – това е "Стоп битът" (Stop bit), който маркира края на рамката.

Този механизъм, макар и сравнително бавен в сравнение със съвременните мрежови протоколи, е изключително устойчив на смущения на къси разстояния (по печатната платка) и изисква минимален брой свързващи проводници (само три – TX, RX и обща земя GND), което го прави златен стандарт за комуникация между модули във вградените системи.

### 3. АПАРАТНО ОСИГУРЯВАНЕ (ХАРДУЕРНА РЕАЛИЗАЦИЯ)

---

Преходът от теоретичните концепции към реално функциониращо устройство изисква внимателно планиране и подбор на хардуерни компоненти, които да работят в пълен синхрон. Физическата реализация на роботизираната платформа е свързана с решаването на редица инженерни предизвикателства – от осигуряването на достатъчна механична стабилност и сцепление, до прецизното управление на енергийните потоци, така че чувствителната електроника да не бъде смущавана от работата на електромоторите. Тази глава разглежда детайлно всеки един от градивните блокове на системата и аргументира направените технически избори.

#### 3.1 ИЗЧИСЛИТЕЛЕН И УПРАВЛЯВАЩ БЛОК

Ядрото на хардуерната архитектура е изчислителният блок, реализиран чрез микроконтролерна платка от фамилията Arduino (напр. Arduino Uno). Този компонент изпълнява ролята на централен процесор на платформата, като неговата основна задача е да служи като мост между комуникационния интерфейс и задвижващата система.

Изборът на Arduino е продиктуван от няколко ключови фактора. На първо място, платката предлага отлична хардуерна абстракция – тя разполага с вграден регулатор на напрежението, хардуерни таймери за генериране на широчинно-импулсна модулация (ШИМ/PWM) и стандартизирани разстояния между пиновете (headers), което позволява лесно надграждане чрез специализирани приставки, наречени шийлдове (shields). В контекста на нашия проект, микроконтролерът работи в непрекъснат цикъл (loop): той "слуша" серийния порт за постъпващи символи от Bluetooth модула, декодира ги мигновено и променя логическите нива на съответните изходи, за да активира желаната посока на движение на моторите.

#### 3.2 ЗАДВИЖВАЩА СИСТЕМА И УПРАВЛЕНИЕ НА ДВИГАТЕЛИ

Един от най-критичните аспекти при проектирането на мобилен робот е задвижващата система. Самият микроконтролер оперира с много малки токове (обикновено до 20-40 mA на пин), които са абсолютно недостатъчни за директно захранване на постояннотоков (DC) двигател. Опит за директно свързване би довел до

моментално изгаряне на микроконтролера. За да се реши този проблем, се използва специализирана схема, известна като Н-мост (H-bridge).

Н-мостът представлява електронна топология от четири превключвателя (транзистори), разположени във формата на буквата "Н", като двигателят е поставен в средата. Чрез отваряне и затваряне на противоположни двойки ключове, схемата позволява обръщане на полярността на напрежението, приложено към мотора, което от своя страна променя посоката на въртене – напред или назад.

В настоящия проект тази логика е реализирана чрез интегрирания драйвер L293D, оформен като Shield приставка. L293D Shield-ът се поставя директно върху Arduino платката, като по този начин драстично се намалява нуждата от сложно окабеляване (кабели "скок" / jumper wires), което често е източник на лоши контакти и хардуерни грешки. Този конкретен драйвер позволява независимото управление на до четири двупосочни DC мотора, като същевременно осигурява вградена защита срещу обратни токове (обратна електродвижеща сила), генерирани от инерцията на моторите при тяхното спиране.

### **Архитектура на шасито: Защо 4 мотора вместо 2?**

По време на фазата на планиране и тестване беше направен важен архитектурен избор относно кинематиката на шасито. Често в любителската роботика се използват платформи с две задвижващи колела (диференциално задвижване) и едно свободно въртящо се сферично колело (caster wheel) за опора. Въпреки че този подход е по-евтин и енергийно по-икономичен, той крие сериозни недостатъци при реална експлоатация.

Използването на шаси с два мотора често води до проблеми със сцеплението и стабилността, особено когато върху платформата са монтирани тежки компоненти като масивните литиево-йонни батерии (18650). Центърът на тежестта може лесно да се измести, което прави робота нестабилен при тръгване или рязко спиране. Поради тази причина, в проекта е заложено използването на четири независими DC мотора (задвижване 4x4). Това решение гарантира отлична симетрия и равномерно разпределение на тежестта. Четирите контактни точки с повърхността осигуряват в пъти по-добро сцепление, предотвратяват приплъзването при завой и позволяват

реализирането на т.нар. танково управление (skid-steering) – роботът може да се завърта на място около собствената си ос чрез въртене на колелата от лявата и дясната страна в противоположни посоки.

### **3.3 ЕНЕРГИЙНА СИСТЕМА И ЗАХРАНВАНЕ**

Енергийното осигуряване на роботизирана система е фундаментален проблем, тъй като механиката и електрониката имат коренно различни изисквания. Двигателите са индуктивни товари, които при стартиране или при среща с препятствие (stall current) консумират пикови стойности на ток. Ако захранването на микроконтролера не е електрически изолирано или добре филтрирано от това на моторите, тези пикови натоварвания могат да предизвикат моментен спад в напрежението (brownout), което води до нежелано рестартиране на Arduino-то и загуба на връзката със сървъра.

За осигуряване на дълъг автономен живот и висок разряден ток са избрани литиево-йонни акумулаторни батерии от стандарт 18650. Те предлагат отлично съотношение между капацитет и тегло. Тъй като номиналното им напрежение (обикновено 3.7V на клетка) варира в зависимост от заряда, последователното им свързване генерира напрежение (напр. 7.4V или 11.1V), което трябва да бъде прецизно регулирано.

За целта в хардуерната конфигурация е интегриран DC-DC преобразувател (понижаващ или buck converter). Неговата роля е да приема променливото високо напрежение от батерийния пакет и да го трансформира в стабилно напрежение, необходимо за оптималната работа на електромоторите и логиката на L293D Shield-a. Това гарантира, че дори при частично разредени батерии, роботът ще запази постоянна скорост и въртящ момент, а управляващата електроника ще остане защитена от флуктуации в мрежата.

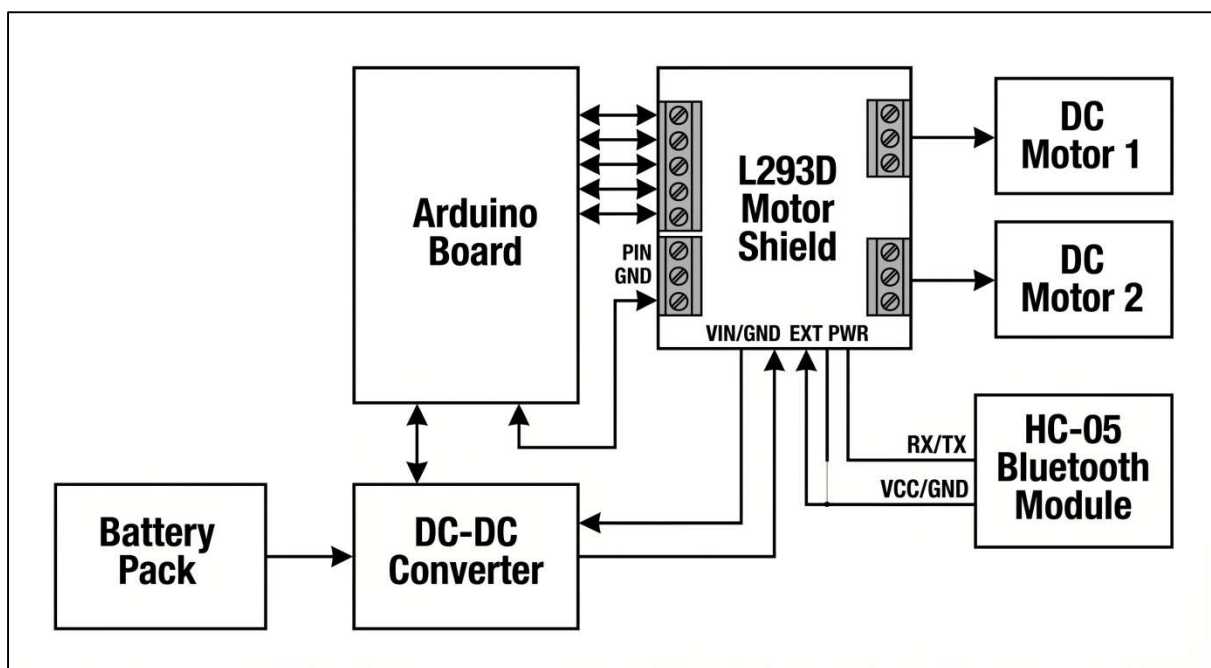
### **3.4 КОМУНИКАЦИОНЕН ИНТЕРФЕЙС И ОБЩА ЕЛЕКТРИЧЕСКА СХЕМА**

Свързването на Bluetooth модула HC-05 към централната система изисква правилно маршрутизиране на сигналите. Модулът комуникира с Arduino чрез сериен интерфейс (UART), използвайки две основни линии: TX (Предаване) и RX (Приемане).

За да се осъществи успешен обмен на данни, TX пинът на Bluetooth модула се свързва към RX пина на микроконтролера, и обратно – RX пинът на модула към TX пина на Arduino.

Важен инженерно-практически детайл тук е съгласуването на напреженията. Докато логическите нива на класическото Arduino обикновено са 5V, комуникационните пинове на HC-05 (по-специално RX пина) са проектирани за 3.3V логика. Директното подаване на 5V сигнал към модула може да съкрати живота му или да доведе до неизправност. Затова добрата инженерна практика изисква използването на резисторен делител на напрежение на линията между предавания пин на Arduino и приемащия пин на HC-05, осигурявайки безопасна комуникация. Захранването на самия модул се осъществява през вградения на платката 5V изход, тъй като HC-05 обикновено разполага със собствен вътрешен регулатор на напрежението за захранващата линия.

На фигурата по-долу е представена общата блокова схема на системата, илюстрираща потока на електрическа енергия и данни между отделните модули.



*ФИГУРА 1: Обща блокова схема на хардуерната архитектура, показваща връзките между Arduino, L293D Shield, DC моторите, DC-DC преобразувателя, батерийния блок и HC-05 Bluetooth модула.*

Цялостният резултат от хардуерното проектиране е компактна, енергийно независима и високо маневрена платформа, която предоставя надеждна основа за имплементацията на софтуерната логика и уеб-базираното управление, разгледани в следващата глава.

## 4. ПРОГРАМНО ОСИГУРЯВАНЕ И РЕАЛИЗАЦИЯ НА СИСТЕМАТА

---

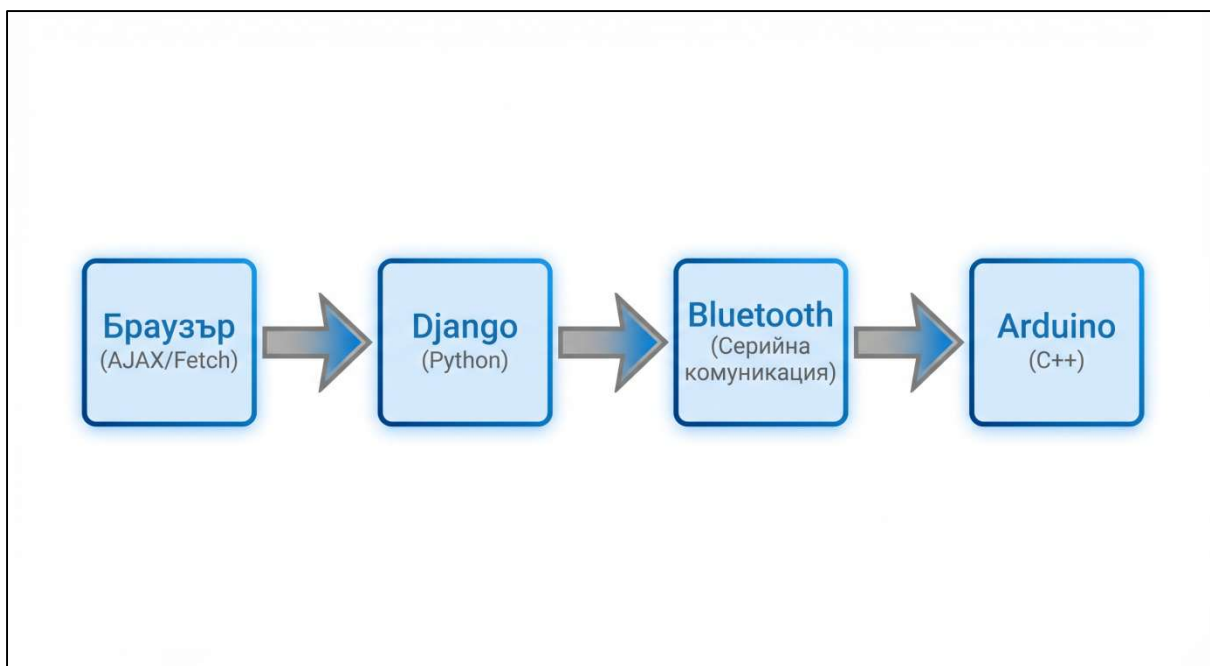
След успешното проектиране и физическо сглобяване на хардуерната платформа, следващата критична фаза от разработването на дипломния проект е програмното осигуряване. Хардуерът сам по себе си представлява единствено физическия потенциал на системата; софтуерът е това, което вдъхва живот на машината, осигурява нейната логика на поведение и реализира моста между човешкото намерение и механичното движение. Тази глава разглежда детайлно многослойната софтуерна архитектура на проекта, като проследява пълния път на една команда – от кликването в уеб браузъра, през обработката ѝ в бекенд сървър, до нейното физическо изпълнение от микроконтролера.

### 4.1 АРХИТЕКТУРА НА СОФТУЕРНАТА СИСТЕМА

Управлението на робот през уеб интерфейс налага използването на разпределена софтуерна архитектура, в която различни технологии комуникират помежду си в реално време. Системата е изградена на принципа на верижна реакция, състояща се от три основни софтуерни възела:

- **Клиентска част (Frontend):** Потребителският интерфейс, зареден в уеб браузър, който улавя действията на потребителя и генерира асинхронни HTTP заявки.
- **Сървърна част (Backend - Django):** Софтуерно приложение, работещо на компютър с Bluetooth модул, което приема HTTP заявките, валидира ги и ги трансформира в серийни данни.
- **Вграден софтуер (Firmware - Arduino):** Програмата, качена в паметта на микроконтролера, която слуша за постъпващи серийни команди и управлява директно драйверите на моторите.





Фигура 2. Архитектурна схема на потока на данни: Браузър (AJAX/Fetch) -> Django (Python) -> Bluetooth (Серийна комуникация) -> Arduino (C++).

Тази модулност гарантира, че всяка част от системата е отговорна за конкретна задача, което улеснява тестването, откриването на грешки (debugging) и бъдещото надграждане.

## 4.2 ПРОГРАМИРАНЕ НА МИКРОКОНТРОЛЕРА (ARDUINO)

Програмният код за микроконтролера е написан на езика C++ в средата за разработка Arduino IDE. Основната цел на този код е да осигури надеждно приемане на данни през хардуерния сериен порт (UART) и тяхното незабавно транслиране в управляващи сигнали за двигателите.

Тъй като комуникацията се осъществява безжично чрез модула HC-05, от ключово значение е да се минимизира обемът на предаваните данни, за да се намали латентността. Поради тази причина бе взето решение командите да не се изпращат като дълги текстови низове (например "MOVE\_FORWARD"), а като единични ASCII символи: 'F' (напред), 'B' (назад), 'L' (наляво), 'R' (надясно) и 'S' (стоп).

За управление на L293D Motor Shield се използва специализирана библиотека (като например AFMotor.h), която абстрахира сложното генериране на ШИМ (PWM) сигнали и улеснява контрола на четирите мотора. По-долу е представен ключов откъс от реализираната логика в микроконтролера:

```
#include <AFMotor.h>
#include <SoftwareSerial.h>

// Инициализация на 4-те мотора
AF_DCMotor motorFrontLeft(2);
AF_DCMotor motorBackLeft(3);
AF_DCMotor motorFrontRight(1);
AF_DCMotor motorBackRight(4);

// Инициализация на софтуерен сериен порт за BLE модула
// RX пин на Arduino (свързан към TX на BLE) -> A4
// TX пин на Arduino (свързан към RX на BLE) -> A5
SoftwareSerial ble(A4, A5);

// Глобална променлива за скоростта (от 0 до 255)
int robotSpeed = 200;

void setup() {
    // Стартиране на серийната комуникация за дебъгване (към компютъра)
    Serial.begin(9600);
    Serial.println("Robot is ready!");

    // Стартиране на комуникацията с BLE модула
    ble.begin(9600);

    // Първоначално задаване на скорост и спиране на моторите
    setSpeedAll(robotSpeed);
    stopRobot();
}
```

Във функцията setup() се инициализира серийната комуникация със скорост 9600 бита в секунда, което е стандартната фабрична настройка на HC-05 модула и е напълно достатъчна за предаване на единични символи без загуба на пакети. Основният цикъл loop() работи непрекъснато, проверявайки хардуерния буфер. Когато пристигне символ, той се подава към функцията executeCommand(), която съдържа логиката за посоките:

```
void loop() {
    // Проверяваме дали има получени данни от BLE модула
    if (ble.available() > 0) {
        char command = ble.read();
        Serial.print("Received command: ");
        Serial.println(command);

        executeCommand(command);
    }
}
```

```

// Функция за обработка на командите
void executeCommand(char cmd) {
    switch (cmd) {
        case 'F': // Напред
            moveForward();
            break;
        case 'B': // Назад
            moveBackward();
            break;
        case 'L': // Наляво
            turnLeft();
            break;
        case 'R': // Надясно
            turnRight();
            break;
        case 'S': // Стоп
            stopRobot();
            break;

        // ДОБАВЕНО УПРАВЛЕНИЕ НА СКОРОСТТА
        case '+':
            robotSpeed += 25; // Увеличаваме скоростта с 25
            if (robotSpeed > 255) robotSpeed = 255; // Ограничаваме до
максимума
            setSpeedAll(robotSpeed); // Прилагаме новата скорост веднага
            Serial.print("Speed Up: ");
            Serial.println(robotSpeed);
            break;

        case '-':
            robotSpeed -= 25; // Намаляваме скоростта с 25
            if (robotSpeed < 0) robotSpeed = 0; // Ограничаваме до минимума
            setSpeedAll(robotSpeed); // Прилагаме новата скорост веднага
            Serial.print("Speed Down: ");
            Serial.println(robotSpeed);
            break;

        default:
            // Игнорираме непознати команди
            break;
    }
}

// Функции за движение
void moveForward() {
    motorFrontLeft.run(FORWARD);
    motorBackLeft.run(FORWARD);
    motorFrontRight.run(FORWARD);
    motorBackRight.run(FORWARD);
}

void moveBackward() {
    motorFrontLeft.run(BACKWARD);
    motorBackLeft.run(BACKWARD);
    motorFrontRight.run(BACKWARD);
    motorBackRight.run(BACKWARD);
}

void turnLeft() {

```

```

    // За завой наляво - левите мотори назад, десните напред
    motorFrontLeft.run(BACKWARD);
    motorBackLeft.run(BACKWARD);
    motorFrontRight.run(FORWARD);
    motorBackRight.run(FORWARD);
}

void turnRight() {
    // За завой надясно - левите мотори напред, десните назад
    motorFrontLeft.run(FORWARD);
    motorBackLeft.run(FORWARD);
    motorFrontRight.run(BACKWARD);
    motorBackRight.run(BACKWARD);
}

void stopRobot() {
    motorFrontLeft.run(RELEASE);
    motorBackLeft.run(RELEASE);
    motorFrontRight.run(RELEASE);
    motorBackRight.run(RELEASE);
}

void setSpeedAll(int speed) {
    motorFrontLeft.setSpeed(speed);
    motorBackLeft.setSpeed(speed);
    motorFrontRight.setSpeed(speed);
    motorBackRight.setSpeed(speed);
}

```

При имплементирането на завоите (наляво и надясно) е използван принципът на "танковото управление" (skid-steering), споменат в Глава 3. Чрез завъртане на левите и десните колела в противоположни посоки, платформата може да се завърти на място, което драстично намалява радиуса на завой и подобрява маневреността в тесни пространства.

### 4.3 РАЗРАБОТКА НА БЕКЕНД СИСТЕМАТА С DJANGO

Сървърната част е реализирана с помощта на уеб рамката Django, базирана на езика Python. Изборът на Django е продиктуван от неговата надеждност, вградени механизми за сигурност и лесната интеграция с библиотеки от по-ниско ниво за достъп до хардуера на компютъра. Основната задача на бекенда е да играе ролята на транслятор между уеб пространството и физическия Bluetooth хардуер на компютъра-сървър.

За осъществяване на комуникацията през Bluetooth (който операционната система вижда като виртуален сериен порт - COM порт при Windows или /dev/rfcomm при Linux), е използвана библиотеката pyserial.

В архитектурата на Django е създаден специален изглед (View), който обработва HTTP заявките от браузъра. По-долу е представен откъс от Python кода, показващ реализацията на този контролер:

```
from django.shortcuts import render
import asyncio
import threading
import queue
from bleak import BleakClient
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

ROBOT_MAC_ADDRESS = "36:33:AB:AE:EB:AB"
WRITE_UUID = "0000ffe1-0000-1000-8000-00805f9b34fb"

# Опащка, в която уеб сървърът ще пуска командите
command_queue = queue.Queue()

# Асинхронна функция, която работи постоянно на заден фон
async def ble_worker():
    while True:
        try:
            print("Опит за свързване с робота на заден фон...")
            async with BleakClient(ROBOT_MAC_ADDRESS) as client:
                print("--- РОБОТЪТ Е СВЪРЗАН! Готов за мигновени команди. ---")

            # Докато е свързан, проверяваме опашката много бързо
            # (на всеки 50ms)
            while client.is_connected:
                try:
                    # Взимаме команда от опашката (без да
                    # блокираме)
                    cmd = command_queue.get_nowait()

                    # Изпращаме веднага
                    command_bytes = cmd.encode('utf-8')
                    await client.write_gatt_char(WRITE_UUID,
                    command_bytes)

                    print(f"Изпратено към робота: {cmd}")

                except queue.Empty:
                    # Ако няма нова команда, изчакваме малко и
                    # въртим цикъла
                    await asyncio.sleep(0.05)

            except Exception as e:
                print(f"Bluetooth прекъсна или не е намерен ({e}). Нов
                опит след 2 секунди...")
                await asyncio.sleep(2)

# Функция за стартиране на фоновата нишка
def start_ble_thread():
    loop = asyncio.new_event_loop()
    asyncio.set_event_loop(loop)
```

```

loop.run_until_complete(ble_worker())

# Стартираме нишката само веднъж при зареждане на файла
ble_thread = threading.Thread(target=start_ble_thread, daemon=True)
ble_thread.start()

class RobotControlView(APIView):
    def post(self, request):
        command = request.data.get('command')
        valid_commands = ['F', 'B', 'L', 'R', 'S', '+', '-']

        if command in valid_commands:
            # Вече НЕ се свързваме тук. Просто слагаме командата в
            # опашката.
            # Това отнема 0.0001 секунди и уеб заявката приключва
            # веднага.
            command_queue.put(command)

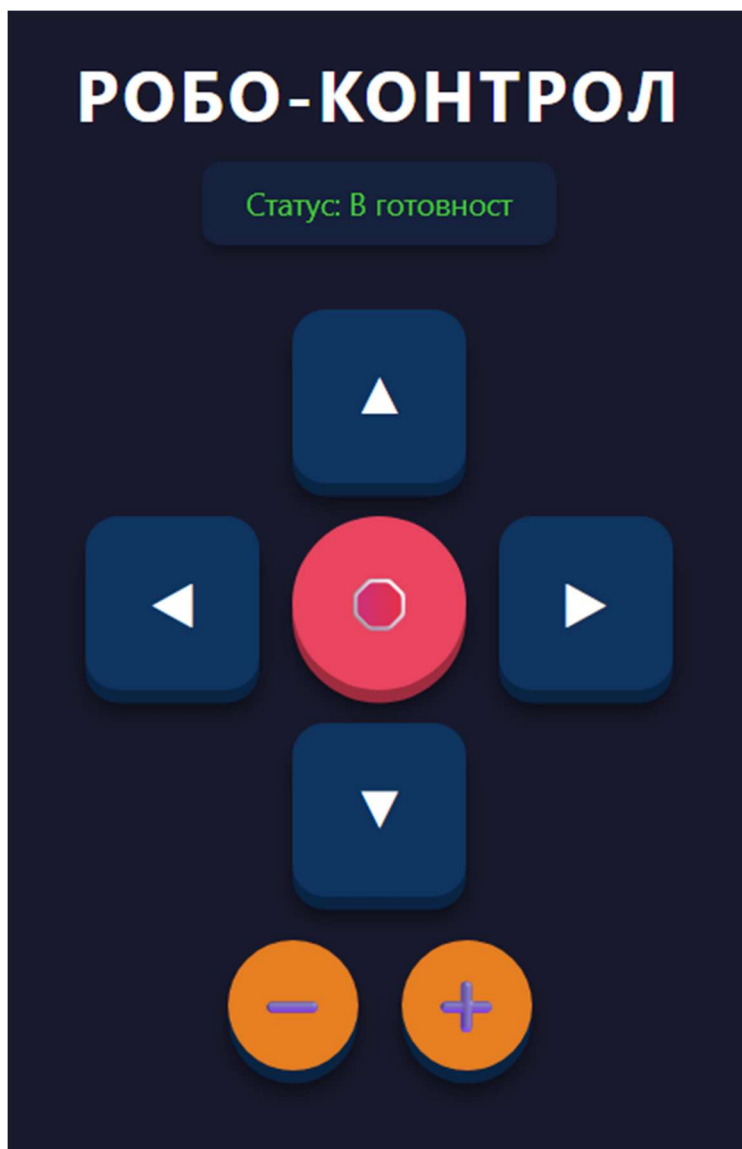
            return Response(
                {"status": "success", "message": f"Команда {command}
изпратена"},
                status=status.HTTP_200_OK
            )
        else:
            return Response(
                {"status": "error", "message": "Грешна команда"},
                status=status.HTTP_400_BAD_REQUEST
            )

```

Този код демонстрира важна концепция в софтуерното инженерство – обработката на изключения (error handling). Ако Bluetooth модулът на робота не е включен или е извън обхват, сървърът няма да се срина, а елегантно ще върне JSON отговор с грешка, който може да бъде обработен от потребителския интерфейс. Преобразуването на низа в байтове (.encode('utf-8')) е задължителна стъпка, тъй като серийният порт оперира на ниво байтове, а не с абстрактни текстови обекти.

## 4.4 ПРОЕКТИРАНЕ НА ПОТРЕБИТЕЛСКИЯ УЕБ ИНТЕРФЕЙС (FRONTEND)

Потребителският интерфейс е визуалната обвивка на проекта, чрез която операторът взаимодейства със системата. Той е проектиран с фокус върху интуитивността и адаптивността (responsive design), за да може роботът да бъде управляван удобно както от компютърна мишка, така и чрез сензорен екран на мобилен телефон.



Изглед на графичния потребителски интерфейс в уеб браузър. Виждат се контролните бутони за управление (Напред, Назад, Наляво, Надясно, Стоп) и индикаторът за състоянието на връзката.

В основата на клиентската логика е залегнала технологията AJAX (Asynchronous JavaScript and XML), реализирана чрез съвременния Fetch API. Традиционните уеб страници презареждат цялото си съдържание при всяко изпращане на данни към сървъра. В контекста на дистанционното управление това е недопустимо, тъй като би създавало огромно закъснение и

би накъсало визуалната обратна връзка. Чрез Fetch API, командите се изпращат във фонов режим.

Следният JavaScript код илюстрира как браузърът улавя натискането на бутон и изпраща командата към Django сървъра:

```
// Взимаме CSRF токена от скритото поле, генерирано от Django
const getCSRFToken = () => {
  return
  document.querySelector('[name=csrfmiddlewaretoken]').value;
};

const statusBar = document.getElementById('status-bar');

// Функция за изпращане на команди към Django бекенда
function sendCommand(cmd) {
  // Визуален фийдбек
  statusBar.innerText = `Изпращане на команда: ${cmd}...`;
}
```

```

statusBar.style.color = '#fbc531'; // Жълт цвят за зареждане

fetch('/api/control/', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-CSRFToken': getCSRFToken() // Добавяме токена в хедъра
  },
  body: JSON.stringify({ command: cmd })
})
.then(response => {
  if (response.ok) {
    return response.json();
  }
  throw new Error('Грешка в мрежата');
})
.then(data => {
  // Успешен отговор
  statusBar.innerText = `Статус: ${data.message}`;
  statusBar.style.color = '#4cd137'; // Зелен цвят за успех
})
.catch(error => {
  // Грешка
  statusBar.innerText = 'Статус: Грешка при връзката сървър!';
  statusBar.style.color = '#e84118'; // Червен цвят за грешка
  console.error('Error:', error);
});
}
// Добавяме клавиатурен контрол (Стрелки за движение, Space за Стоп)
document.addEventListener('keydown', function(event) {
  switch(event.key) {
    case 'ArrowUp': sendCommand('F'); break;
    case 'ArrowDown': sendCommand('B'); break;
    case 'ArrowLeft': sendCommand('L'); break;
    case 'ArrowRight': sendCommand('R'); break;
    case ' ': sendCommand('S'); break;
    case '+': case '=': sendCommand('+'); break;
    case '-': sendCommand('-'); break;
  }
});

```

Особен интерес в този код представлява имплементирането на събитията `onclick` (натискане на бутон), което обвързва графичния интерфейс и формирането на управляващи команди:

```

<div class="d-pad">
  <button class="btn btn-up" onclick="sendCommand('F')">▲</button>
  <button class="btn btn-left"
onclick="sendCommand('L')">◀</button>
  <button class="btn btn-stop"
onclick="sendCommand('S')">●</button>
  <button class="btn btn-right"
onclick="sendCommand('R')">▶</button>
  <button class="btn btn-down"
onclick="sendCommand('B')">▼</button>
</div>
<div class="speed-controls">
  <button class="btn btn-speed" onclick="sendCommand('-

```



```
' ) "></button>  
    <button class="btn btn-speed"  
onclick="sendCommand('+' ) "></button>  
</div>
```

## 5. ТЕСТВАНЕ, АНАЛИЗ И РЕЗУЛТАТИ

---

След приключване на хардуерното сглобяване и софтуерното програмиране, системата премина през серия от тестови процедури. Целта на този етап е не просто да се докаже, че роботът "работи", а да се анализират неговото поведение в реални условия, стабилността на комуникацията и границите на неговата надеждност. Тестването на комплексни системи изисква методичен подход, при който всеки модул се проверява първо изолирано, а след това и като част от общото цяло.

### 5.1 МЕТОДИКА НА ТЕСТВАНЕ

Процесът на тестване беше разделен на три основни фази: хардуерна валидация, проверка на комуникационния канал и полеви изпитания на интегрираната система.

Първата фаза започна с "тестване на сухо" (dry run) на хардуера. Преди микроконтролерът да бъде свързан към захранването, всички електрически вериги бяха проверени за къси съединения посредством мултицет. Особено внимание беше обърнато на изхода от DC-DC преобразувателя, за да се гарантира, че подаваното към L293D Shield-а и Arduino-то напрежение е в абсолютно точните граници (около 5V до 7V, в зависимост от входния пин), независимо от нивото на заряд на 18650 батериите. След това роботът беше повдигнат на стенд (без колелата да докосват земята) и беше качен базов тестов код, който просто завърта четирите мотора последователно. Тази стъпка потвърди, че драйверът функционира правилно и че всички двигатели са свързани с правилната полярност.

Втората фаза се фокусира върху комуникацията. Тук целта беше да се изолира софтуерният стек от хардуерната механика. Чрез използване на инструмента Serial Monitor в Arduino IDE, бяха симулирани входящи Bluetooth команди. След успешното

им разчитане, системата беше свързана с Django сървър. Чрез конзолата на брауъра (Developer Tools) се проследяваха генерираните AJAX заявки при натискане на бутоните в уеб интерфейса, а в терминала на сървъра се наблюдаваше дали съответните символи се изпращат коректно към виртуалния COM порт на Bluetooth модула HC-05.

Третата фаза включваше интегрирани полеве тестове. Роботът беше поставен на различни повърхности (гладък под, килим, лек наклон), за да се оцени ефективността на задвижващата система 4x4 при реално съпротивление. Управляван директно през брауъра на мобилен телефон, свързан към същата локална мрежа като Django сървър, роботът изпълняваше пълния набор от команди – напред, назад, леви и десни завои на място.

## **5.2 АНАЛИЗ НА НАДЕЖНОСТТА И ЗАКЪСНЕНИЯТА (LATENCY)**

Един от най-важните параметри при системите за дистанционно управление е латентността – времето, което изтича от момента на подаване на командата от потребителя до реалното физическо движение на машината. В нашата архитектура сигналът преминава през няколко "скока" (hops): от брауъра през Wi-Fi мрежата до Django сървър, след това от сървър през Bluetooth протокола до HC-05 модула, и накрая през серийния буфер до процесора на Arduino.

По време на тестовете беше установено, че при нормални условия на локална мрежа (LAN), общото закъснение е в рамките на 100 до 250 милисекунди. Тази латентност е практически незабележима за човешкото око и осигурява усещане за управление в реално време. Въпреки това, в бележките по време на разработката бяха идентифицирани потенциални тесни места при комуникацията с HC-05. Ако Bluetooth връзката бъде смутена от външни радиочестотни интерференции или ако роботът доближи границата на обхвата си (около 10 метра), се наблюдават временни закъснения (лаг) или загуба на пакети.

Точно тук се проявява критичната важност на софтуерния дизайн, обсъден в Глава 3 – по-специално изпращането на команда "Стоп" ('S') при отпускане на виртуалния бутон. Ако роботът се движи и връзката временно прекъсне, той може да продължи да изпълнява последната получена команда. За да се оптимизира сигурността в бъдещи версии, може да се имплементира хардуерен "watchdog" таймер в Arduino –

ако микроконтролерът не получи никакъв сигнал (дори потвърждаващ "ping") в продължение на повече от 1 секунда, той автоматично да спира моторите.

### 5.3 ОЦЕНКА НА ЦЯЛОСТНОТО ПРЕДСТАВЯНЕ

Резултатите от полевите тестове категорично доказват правилността на взетите инженерни решения. Изборът на шаси с четири независими DC мотора се оказа изключително успешен. За разлика от платформи с две задвижващи колела, които често губят сцепление или се отклоняват от правата линия при неравности, реализираната 4x4 система демонстрира висока курсова стабилност. Танковото управление (skid-steering) позволява на платформата да маневрира в изключително тесни пространства, като центърът на завъртане съвпада с геометричния център на работа.



**Arduino Uno**



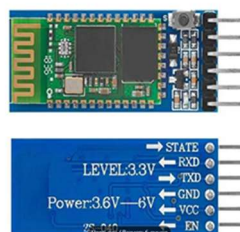
**L293D**



**DC-DC converter**



**DC motors and wheels**



**HC-05 Bluetooth**

*Снимка 1: Електронни компоненти*

Енергийната система, базирана на литиево-йонни клетки 18650 и DC-DC преобразувател, осигури повече от достатъчен ток за преодоляване на стартовото съпротивление на моторите. По време на интензивни тестове не бяха регистрирани случаи на рестартиране на микроконтролера поради пад на напрежението, което

потвърждава успешната електрическа изолация между логическата и силовата част на схемата.



*Снимка 1: Общ изглед на напълно сглобената роботизирана платформа.*

## 6. ЗАКЛЮЧЕНИЕ

---

Настоящият дипломен проект успешно реализира поставената цел – създаването на пълноценна, дистанционно управлявана роботизирана платформа чрез веб интерфейс. Изградената система демонстрира успешната интеграция на хардуерни компоненти на ниско ниво (микроконтролери, моторни драйвери и сензори) със съвременни софтуерни веб технологии от високо ниво.

Проектът изпълни всички функционални изисквания, заложи в заданието. Разработеният веб интерфейс предоставя интуитивен контрол върху основните посоки на движение (напред, назад, наляво, надясно и спиране), като обратната връзка за състоянието на системата се визуализира в реално време на екрана на потребителя. Използването на клиент-сървърна архитектура с базиран на Python/Django бекенд и асинхронен JavaScript фронтенд доказва своята ефективност при транслирането на потребителски намерения във физически действия. От хардуерна гледна точка, стабилността на четирите мотора и надеждното хранване гарантират предвидимо и безопасно поведение на машината.

Възможности за бъдещо развитие:

- Макар системата да е напълно функционална в настоящия си вид, нейната модулна архитектура предоставя отлична основа за бъдещи надграждания. Някои от най-перспективните насоки за развитие включват:
- Интеграция на сензори за автономност: Добавяне на ултразвукови сензори (например HC-SR04), които да измерват разстоянието до препятствия и автоматично да спират робота преди сблъсък, добавяйки слой на хардуерна безопасност, независим от оператора.
- Видео стрийминг в реално време: Монтиране на IP камера върху шасито, чиято картина да се вгради директно в веб интерфейса на Django сървър, позволявайки управление извън зрителното поле на оператора (FPV управление).
- Миграция към Wi-Fi базиран микроконтролер: В бъдеще, Arduino и HC-05 модулът могат да бъдат заменени с микроконтролер с вграден Wi-Fi модул (като ESP32). Това би елиминирало нуждата от междинен компютър, свързан през Bluetooth, като самият робот ще се свързва директно към локалната мрежа и ще хоства своя собствен мини-сървър.

В заключение, проектът не само затвърждава теоретичните знания в областта на електрониката, програмирането и компютърните мрежи, но и предоставя ценен практически опит в решаването на реални инженерни проблеми при създаването на кибер-физични системи.

## 7. ИЗПОЛЗВАНА ЛИТЕРАТУРА И ИЗТОЧНИЦИ

---

1. Документация на Arduino (Arduino Reference): Официално ръководство за програмиране на микроконтролери и работа с хардуерни интерфейси ([www.arduino.cc](http://www.arduino.cc)).
2. Документация на Django (Django Web Framework): Официална документация за изграждане на уеб приложения, маршрутизиране и управление на сървърни процеси ([docs.djangoproject.com](http://docs.djangoproject.com)).
3. Техническа спецификация на HC-05: Ръководство за конфигурация и работа с Bluetooth SPP (Serial Port Protocol) модули.
4. Спецификация на L293D: Информационен лист (Datasheet) за интегрирани драйвери за управление на постояннотокови двигатели.
5. Документация на PySerial: Ръководство за работа със серийни портове чрез езика за програмиране Python ([pyserial.readthedocs.io](http://pyserial.readthedocs.io)).
6. Mozilla Developer Network (MDN Web Docs): Референтни материали за работа с Fetch API, асинхронен JavaScript и управление на събития (DOM Events) в браузъра.